GUÍA DE EXPOSICIÓN • SISTEMA DE GESTIÓN Y VENTA DE CURSOS

Arquitectura de Microservicios en Spring Boot 3 • PostgreSQL • QA con Postman • Estrategia Frontend

1. Base de Datos (PostgreSQL)

¿QUÉ DECIR EN LA EXPOSICIÓN?

"Usamos PostgreSQL con un modelo relacional normalizado de 5 entidades. Diseñamos la base de datos priorizando la seguridad y la integridad estricta mediante UUIDs y restricciones CHECK."

- **Uso de UUIDs (gen_random_uuid()):** No usamos IDs numéricos secuenciales por seguridad (evita ataques de enumeración) y para soportar arquitectura distribuida.
- **Integridad y Restricciones CHECK:** aforo_disponible >= 0 (previene sobreventa a nivel BD); si la modalidad es VIRTUAL se exige enlace (Meet/Zoom), y si es PRESENCIAL se exige aula/dirección física.
- **Tablas Principales:** usuarios (Roles ADMIN, DOCENTE, ESTUDIANTE y 2FA), cursos (aforos, precios, docentes), pedidos (órdenes ORD-..., Mercado Pago), comprobantes (Boleta/Factura con IGV 18%) y notificaciones (WhatsApp/Email).

2. Aseguramiento de Calidad (QA & Testing)

¿QUÉ DECIR EN LA EXPOSICIÓN?

"Nuestra estrategia de QA valida cada nivel: validación de contratos en los DTOs, manejo centralizado de excepciones y una suite completa de pruebas de integración E2E en Postman."

- **Validación de Entradas (Jakarta Validation):** Uso de @NotBlank, @Email, @NotNull y @Valid en DTOs para bloquear datos inválidos antes de procesar lógica de negocio.
- **Manejo Global de Excepciones:** Controlador centralizado (@RestControllerAdvice / ApiExceptionHandler) retornando códigos HTTP estándar uniformes (400, 404, 409).
- **Pruebas de Integración con Postman:** Colección modular oficial con variables de entorno probando el flujo completo: *Registro/Login 2FA → Catálogo vía Gateway → Checkout con reserva temporal (15 min) → Pago y emisión de comprobante*.

3. Estrategia y Planificación de Frontend

¿QUÉ DECIR EN LA EXPOSICIÓN?

"El backend está 100% desacoplado. El frontend se diseñó como una SPA (React/Angular/Vue) que se comunica exclusivamente con el API Gateway en el puerto 8080 sin conocer los puertos internos."

- **Punto de Entrada Único (API Gateway):** El Frontend solo apunta a http://localhost:8080/api/..., simplificando CORS y la seguridad perimetral.
- **Flujo de Usuario por Rol en la UI:**
  - **Estudiante:** Catálogo interactivo, checkout con contador regresivo de reserva y panel 'Mis Cursos' con botón directo a Zoom/Meet.
  - **Docente:** Pantalla de desafío OTP para 2FA y panel de gestión de alumnos inscritos en tiempo real.
  - **Administrador:** Mantenimiento de cursos, asignación de docentes y reportes de facturación.
- **Seguridad:** Autenticación vía Token Bearer en headers HTTP interceptados automáticamente por Axios/Fetch.

4. Arquitectura y Puertos de Microservicios

Microservicio	Puerto	Responsabilidad Principal	Interacción
gateway-service	**8080**	Enrutamiento centralizado y seguridad perimetral	Punto único de entrada
auth-service	**8081**	Usuarios, Roles y Autenticación OTP 2FA	PostgreSQL :5433
cursos-service	**8082**	Catálogo de cursos, aforos y enlaces virtuales	PostgreSQL :5433
pedidos-service	**8083**	Checkout, reserva temporal, pagos, boletas y WhatsApp	Consume Auth y Cursos

■ DISCURSO DE CIERRE / PITCH DE 1 MINUTO (Para memorizar)

"En conclusión, nuestro proyecto presenta una arquitectura de microservicios robusta y escalable desarrollada en Spring Boot 3 y PostgreSQL. Garantizamos seguridad mediante 2FA, alta integridad con UUIDs y reglas de aforo a nivel de base de datos, aseguramiento continuo con Postman en la capa de QA, y una API REST desacoplada y lista para integrarse de inmediato con cualquier frontend moderno."